Compiled By: Saom Bin Khaled [ID: 220041224] [March 6, 2025]

SQL Commands Cheat Sheet

1. Preprocessing

```
CONNECT SYS/SYS AS SYSDBA;
CREATE USER username IDENTIFIED BY password;
GRANT ALL PRIVILEGES TO username;
CONNECT username/password;
CL SCR;
```

2. Create Table

```
CREATE TABLE table_name (
attribute1 datatype [ NULL | NOT NULL | UNIQUE ],
attribute2 datatype [ NULL | NOT NULL | UNIQUE ],
...,
[CONSTRAINT constraint_name] PRIMARY KEY (primary_attribute1, ...),
[CONSTRAINT constraint_name] CHECK condition
);
```

3. Drop Table

```
DROP TABLE table_name;
DROP TABLE table_name CASCADE CONSTRAINTS;
```

4. Alter Table

```
ALTER TABLE table_name ADD (attribute1 datatype, ...);
ALTER TABLE table_name DROP COLUMN (attribute1, ...);
ALTER TABLE table_name MODIFY attribute_name new_datatype;
ALTER TABLE table_name RENAME COLUMN old_attribute_name TO new_attribute_name;
ALTER TABLE table_name RENAME TO new_table_name;
```

5. Insert Into Table

```
INSERT INTO TABLE_NAME VALUES (..., ..., ...);
```

6. Update Table

```
UPDATE TABLE_NAME
SET column1 = value1, column2 = value2, ...
WHERE <CONDITIONAL CLAUSE>;
```

7. Delete From Table

```
DELETE FROM TABLE_NAME
WHERE <CONDITIONAL CLAUSE>;
```

8. Retrieve From Table

```
SELECT ATTRIBUTE1, ATTRIBUTE2
FROM TABLE_NAME
WHERE <CONDITIONAL CLAUSE>;

SELECT * FROM TABLE_NAME;
```

9. Execute SQL Script

```
@FILE_PATH_LOCAL_STORAGE
```

10. Create Table with Foreign Key

```
CREATE TABLE table_name (
...,
[ CONSTRAINT constraint_name ] FOREIGN KEY (foreign_attribute1, ...)
REFERENCES reference_table_name
[ON DELETE CASCADE | SET NULL | SET DEFAULT]
[ON UPDATE CASCADE | SET NULL | SET DEFAULT]
);
```

11. DISTINCT

```
SELECT DISTINCT attributename FROM tablename;
```

12. BETWEEN - Range Operations

```
SELECT attribute_1, ...  FROM tablename
WHERE attribute_1 BETWEEN l_value AND u_value; [INCLUSION]
WHERE attribute_1 NOT BETWEEN l_value AND u_value; [EXCLUSION]
```

13. IN - Multiple Values in WHERE Clause

```
SELECT attribute_1, ...  FROM tablename
WHERE attribute_1 IN (value1, value2, ...); [Exact matching]
```

14. String matching, Concatenation

```
SELECT attribute_1, attribute_2, ...  FROM tablename
WHERE attribute_1 LIKE pattern; ['%' = any substring '_' = single character]
'||' = concatanetion
```

15. Join Between Tables

```
SELECT column1, column2, ...  FROM table1
JOIN table2 ON table1.column = table2.column;
```

16. Aggregate Functions

```
SELECT [AVG | MIN | MAX | SUM | COUNT](attribute_1) FROM EMPLOYEE;
```

17. GROUP BY Clause Rule

All selected attributes must be present either in aggregate functions
or in the GROUP BY clause, otherwise, the query will be considered erroneous.

18. HAVING Clause Rule

As was the case for the SELECT statement, any attribute that
is present in the HAVING clause without being aggregated must
appear in the GROUP BY clause, otherwise, the query is treated
as erroneous.

HAVING is used over GROUP BY clause conditions

Example:

```
SELECT DEPT , AVG ( SALARY ) FROM EMPLOYEE
GROUP BY DEPT
HAVING AVG ( SALARY ) > 5000;
```

19. DATE Functions

```
1.  CURRENT DATE and SYSDATE
SELECT CURRENT_DATE FROM DUAL;
SELECT SYSDATE FROM DUAL;

2.  TO_DATE - Convert String to Date
SELECT TO_DATE('20 APR 2020', 'DD MON YYYY') AS CONVERTED_DATE FROM DUAL;

3.  TO_CHAR - Convert Date to String
SELECT TO_CHAR(SYSDATE, 'DD-MM-YYYY') AS NEW_DATE FROM DUAL;

4.  EXTRACT - Extract Year, Month, or Day
SELECT EXTRACT(YEAR FROM SYSDATE) AS YEAR FROM DUAL;

5.  LAST_DAY - Get Last Day of the Month
SELECT LAST_DAY(SYSDATE) AS LAST_DAY FROM DUAL;

6.  NEXT_DAY - Get Next Occurrence of a Weekday
SELECT NEXT_DAY(SYSDATE, 'FRIDAY') FROM DUAL;

7.  MONTHS_BETWEEN - Difference Between Two Dates in Months
SELECT MONTHS_BETWEEN(SYSDATE, DATE '2011-04-02') AS MONTH_DIFFERENCE FROM DUAL;

8.  ADD_MONTHS - Add N Months to a Date
SELECT ADD_MONTHS(SYSDATE, 2) AS NEW_DATE FROM DUAL;

9.  CURRENT_TIMESTAMP - Get Current Date and Time with Timezone
SELECT CURRENT_TIMESTAMP FROM DUAL;

10.  SYSTIMESTAMP - Get System Timestamp
SELECT SYSTIMESTAMP FROM DUAL;
```

```
11. TRUNC - Remove Time Portion from a Date
SELECT TRUNC(SYSDATE) AS TRUNCATED_DATE FROM DUAL;

12. ROUND - Round Date to Nearest Month or Year
SELECT ROUND(SYSDATE, 'MONTH') AS ROUNDED_DATE FROM DUAL;

13. NEW_TIME - Convert Time Between Time Zones
SELECT NEW_TIME(SYSDATE, 'EST', 'PST') AS CONVERTED_TIME FROM DUAL;
```

20. STRING Functions

```
1. LENGTH - Get String Length
SELECT LENGTH('HELLO') FROM DUAL;

2. LOWER - Convert to Lowercase
SELECT LOWER('Hello') FROM DUAL;

3. UPPER - Convert to Uppercase
SELECT UPPER('Hello') FROM DUAL;

4. INITCAP - Capitalize First Letter of Each Word
SELECT INITCAP('hello world') FROM DUAL;

5. TRIM - Remove Leading/Trailing Characters
-- Trim both leading and trailing spaces
SELECT TRIM(' Hello World ') AS NEW_STRING FROM DUAL;
-- Trim leading character '6'
SELECT TRIM(LEADING '6' FROM '665530') AS NEW_STRING FROM DUAL;

6. LPAD - Left Padding
SELECT LPAD('Hello', 10, '+') AS NEW_STRING FROM DUAL;

7. RPAD - Right Padding
SELECT RPAD('Hello', 10, '+') AS NEW_STRING FROM DUAL;

8. REPLACE - Replace Characters in a String
SELECT REPLACE('JACK with JUE', 'J', 'BL') AS NEW_STRING FROM DUAL;

9. SUBSTR - Extract a Portion of a String
SELECT SUBSTR('Hello World', 1, 5) AS SUBSTRING FROM DUAL;

10. INSTR - Find Position of a Substring
SELECT INSTR('Hello World', 'o') AS POSITION FROM DUAL;

11. CONCAT - Concatenate Two Strings
SELECT CONCAT('Hello', ' World') AS FULL_STRING FROM DUAL;

12. TRANSLATE - Replace Multiple Characters
SELECT TRANSLATE('HELLO', 'HE', 'YX') AS NEW_STRING FROM DUAL;
```

21. Handling NULL Values

```
1. NVL - Replace NULL with a Default Value
SELECT NVL(column_name, replacement_value) FROM table_name;

2. NVL2 - Different Values for NULL and NOT NULL
SELECT NVL2(commission, 'Has Commission', 'No Commission') FROM sales;
```

3. COALESCE – Return First Non-NULL Value

```
SELECT COALESCE(supplier_city, region, 'Unknown') FROM suppliers;
```

4. NULLIF – Returns NULL if Two Values Match

```
SELECT NULLIF(salary, 5000) FROM employees;
```

22. CASE Statement

```
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    ELSE default_result
END
```

Example – Using CASE in an UPDATE Statement:

```
UPDATE ORDER_ITEM OI
SET STATUS = (
    SELECT CASE
        WHEN D.PRICE > (SELECT AVG(PRICE) FROM DISH) + 5 THEN 'Expensive'
        WHEN D.PRICE < (SELECT AVG(PRICE) FROM DISH) - 5 THEN 'Cheap'
        ELSE 'Average'
    END
    FROM DISH D
    WHERE OI.DISH_ID = D.DISH_ID
);
```

PL/SQL

23. Block Structure

```
SET SERVEROUTPUT ON;
DECLARE
    -- Declaration statements (variables, constants, cursors)
BEGIN
    -- Executable statements (SQL queries, loops, conditions)
EXCEPTION
    -- Exception handling statements (handling runtime errors)
END;
/
```

24. Conditionals – Check Even or Odd

```
DECLARE
    num NUMBER := 7;
BEGIN
    IF MOD(num, 2) = 0 THEN
        DBMS_OUTPUT.PUT_LINE('The number is even.');
    ELSE
        DBMS_OUTPUT.PUT_LINE('The number is odd.');
    END IF;
END;
/
```

25. Function

```
CREATE FUNCTION function_name (parameter_name datatype, ...)
RETURN return_datatype
IS
    -- Variable declarations
    variable_name datatype;
BEGIN
    -- Function logic (SQL queries, calculations, conditions)

    RETURN return_value; -- The function must return a value
EXCEPTION
    -- Exception handling (optional)
    WHEN others THEN
        RETURN error_value;
END function_name;
/
```

Example:

```
CREATE FUNCTION get_increment(emp_id NUMBER)
RETURN NUMBER
IS
    emp_salary NUMBER;
    increment NUMBER;
BEGIN
    -- Get employee salary
    SELECT salary INTO emp_salary FROM employees WHERE employee_id = emp_id;

    -- Apply increment logic
    IF emp_salary > 5000 THEN
        increment := emp_salary * 0.10; -- 10% increment
    ELSE
        increment := emp_salary * 0.20; -- 20% increment
    END IF;

    RETURN increment;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 0; -- Return 0 if employee not found
END get_increment;
/
```

Calling the Function:

```
SELECT get_increment(101) FROM dual;
```

26. Procedure

```
CREATE PROCEDURE procedure_name (param1 IN datatype, param2 OUT datatype, ...)
IS
    -- Variable declarations
    variable_name datatype;
BEGIN
    -- Executable statements (SQL queries, calculations, conditions)
    -- Use param1 and set values for OUT parameters
EXCEPTION
    -- Exception handling (optional)
    WHEN others THEN
        -- Handle errors
END procedure_name;
/
```

Example: A Procedure to Find the Minimum of Two Numbers

```
CREATE PROCEDURE findMin (x IN NUMBER, y IN NUMBER) IS
    z NUMBER; -- Declare the variable z
BEGIN
    IF x < y THEN
        z := x;
    ELSE
        z := y;
    END IF;
    dbms_output.put_line('Minimum is: ' || z);
END;
/
```

Executing the Procedure:

```
DECLARE
    a NUMBER;
    b NUMBER;
BEGIN
    a := 23;
    b := 45;
    findMin(a, b);
END;
/
```

27. Loops

WHILE Loop:

```
DECLARE
    -- Variable declarations
BEGIN
    WHILE condition LOOP
        -- Statements to execute
    END LOOP;
END;
/
```

FOR Loop:

```
BEGIN
    FOR loop_variable IN start_value..end_value LOOP
        -- Statements to execute
    END LOOP;
END;
/
```